

实验 A4 Vivado 软件的使用(包括仿真)

一、概述

1. Xilinx 的 FPGA 的开发设计流程

Xilinx 的 FPGA 的开发设计流程, 如图 A4.18 所示, 包括设计输入、功能仿真、综合、实现、时序仿真、下载配置和在线调试等芯片编程与调试等主要步骤。

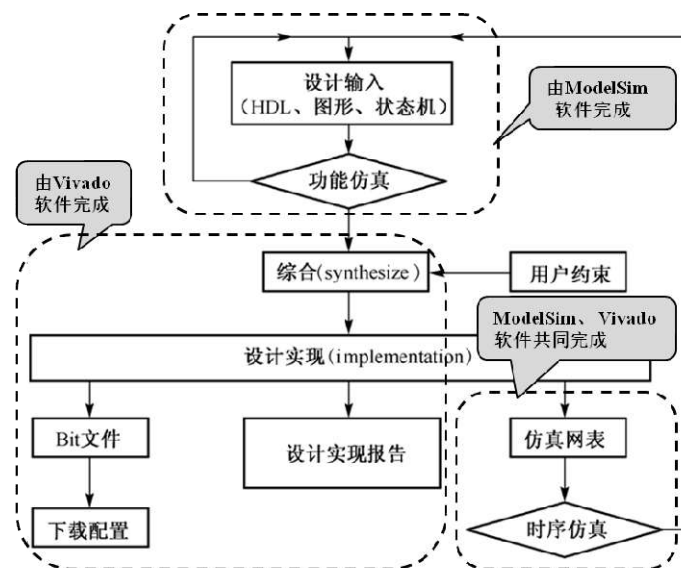


图 A4.18 Xilinx 的 FPGA 开发流程图

1) 设计输入 (design entry)

Xilinx 的 FPGA 的开发软件 Vivado 采用基于工程的分层次管理, 支持硬件描述性语言 (HDL)、逻辑图和状态图等混合输入方式。本教程只采用 Verilog HDL 硬件描述性语言进行设计输入, 因此设计可采用 ModelSim 或 Vivado 软件的编辑器, 也可采用 Notepad++、UltraEdit 等专业代码编辑软件, 建议使用后者。

2) 功能仿真 (behavioral simulation)

功能仿真的主要目的是验证设计文件的逻辑功能是否正确, 即在理想状态下 (不考虑延迟), 验证电路的功能是否符合设计的要求。

3) 用户约束 (user constraints)

在进行高速数字电路设计时, 设计者经常需要对时序、I/O 引脚位置和区域等附加约束, 以便控制综合、实现过程, 使设计满足运行速度、面积和引脚位置等方面的要求。这些约束条件存放在用户约束文件 (.XDC 文件) 中, 用户可随时编辑修改。

4) 综合 (synthesize)

综合是使用 HDL 设计的重要环节, 是将 HDL 描述转换为用基本门电路、RTL 和厂家提供的基本单元进行描述的过程。

5) 设计实现 (implementation)

Vivado 的实现功能包括了逻辑优化(opt_design)、映射 (map)、布局 (place_design) 和布线 (route_design) 等过程。

6) 时序仿真 (timing simulation)

时序仿真是在将设计适配到 FPGA 芯片后的仿真验证方式。时序仿真在严格的仿真时间模型下,模拟芯片的实际运作。仿真时间模型将最基本的门级时延计算在内,从而可有效地分析出设计中的竞争和冒险。在多数情况下,经过时序仿真验证后的设计基本上与实际电路是一致的。

7) 下载配置 (programming)

下载功能包括了 BitGen,用于将布局布线后的设计文件转换为比特流(bitstream)文件。还包括了 iMPACT 功能,用于进行设备配置和通信,控制程序烧写到 FPGA 芯片中。

本教程将用两个实验介绍 FPGA 开发设计流程,在实验 1 介绍功能仿真这一步骤,实验 2 介绍综合、实现、下载配置等开发过程。

2. Vivado 软件简介

Vivado 软件是 FPGA 厂商 Xilinx 公司的集开发工具套件,包括高度集成的设计环境和新一代从系统到 IC 级的开发工具,这些均建立在共享的可扩展数据模型和通用调试环境基础上。这也是一个基于 AMBA AXI4 互联规范、IP-XACT IP 封装元数据、工具命令语言(TCL)、Synopsys 系统约束(SDC) 以及其它有助于根据客户需求量身定制设计流程并符合业界标准的开放式环境。Xilinx 构建的 Vivado 工具把各类可编程技术结合在一起,能够扩展多达 1 亿个等效 ASIC 门的设计。Vivado 软件主要特点有。

1) 完整的设计工具。

设计全部流程完成在一个工具中,综合时就可考虑整个设计网表,在设计各个阶段都需要检查各种约束文件,设计的整体性得到保证。

2) 更优的 PC 资源使用方式。

开发软件是多个工具组合在一起,各工具交接时,产生临时文件来保存上一步的执行结果。Vivado 软件的设计思想是,直接保存在内存中,这避免了反复读写硬盘的麻烦。这一效果,对于越大的设计,体现得更明显。

3) 更好的时序收敛。

Vivado 软件在时序设计、布线延时、资源等多方面时行综合考虑,具有更好的时序收敛。

4) Vivado 软件全面支持 Tcl 脚本。

5) 支持 System Verilog。

二、实验内容

介绍流水灯实验的仿真、综合、实现和下载配置过程。

三、实验设备

(1) 装有 Vivado 软件的计算机。

(2) Nexys Video 开发板或 Basys 3 开发板。

四、设计代码

1. 实验任务

设计并实现一个流水灯电路,要求:

(1) 8 个发光二极管间隔 0.5s 轮流点亮。

(2) 设置一个 direction 开关控制流水灯流水灯点亮方向。direction=0 时,向右轮流点亮; direction=1 时,向左轮流点亮。

(3) 设置一个 reset 按钮,复位时,最左边灯亮。

2. 电路总体框图

根据设计要求，流水灯电路可由三个子模块组成，如图 A4.19 所示。分频器模块产生周期为 0.5 秒的脉冲信号 `pluse2Hz`，该信号决定流水灯的流动速度。计数器模块为三位可逆计数器，该模块决定流水灯的流动方向，`direction=0`，减法计数（向右轮流点亮）；`direction=1`，加法计数（向左轮流点亮）；译码器模块则根据计数状态 `q` 点亮相应编号的发光二极管。

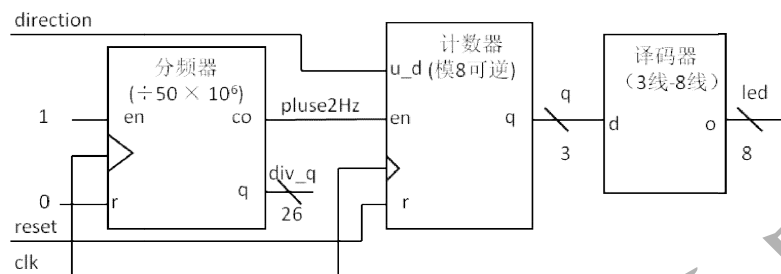


图 A4.19 流水灯的顶层电路连接图

3. Verilog HDL 代码

由于电路具有大分频比的分频器模块，实时仿真基本不可能也没必要。因此，一般做法，仿真时将分频器改为小分频比的分频器模块，以减少仿真时间。具体解决方案，在顶层模块设置参数 `sim`，`sim` 默认值为 0。`sim` 取值决定分频器的分频比：`sim=0`，分频比设置为 50×10^6 ；而 `sim=1`，分频比设置 10。在测试代码，调用 `top` 模块，传递 `sim` 值为 1，这样就减少了仿真时间。另外，顶层模块中的 `sim` 默认 0 值不影响电路综合和实现。

下面，给出实验的部分 Verilog HDL 代码，除了理解电路原理外，读者还应重点理解参数 `sim` 的作用。

（1）流水灯电路的顶层模块

```
module FlowingWaterLight(clk,reset,direction,led);
    parameter sim=0;// 默认综合方式
    input clk,reset,direction;
    output[7:0] led;

    //n 进制计数器/分频器实例，sim=0 时，分频比 n=5000_0000，计数器位数
    //counter_bits=26;而 sim=1 时，分频比 n=10，计数器位数 counter_bits=4。
    //----- 可逆计数器实例 -----//
    wire pulse2Hz;
    counter_n #(n(sim?10:5000_0000),counter_bits(sim?4:26))
        div_inst(.clk(clk),.en(1),.r(0),.q(),.co(pulse2Hz));
    wire[2:0] q;
    counter_up_dn #(counter_bits(3)) counter_up_dn_inst(
        .clk(clk),.en(pulse2Hz),.u_d(direction),.r(reset),.q(q));
    // ----- 3 线-8 线器实例 -----//
    decode decode_inst(.din(q),.out(led));
endmodule
```

（2）n 进制计数器/分频器模块

```
module counter_n(clk,en,r,q,co);
```

```

parameter  n=2; //参数 n 表示计数的模
parameter  counter_bits=1; //参数 counter_bits 表示计数的位数
input  clk,en,r ;
output  co;
output  [counter_bits-1:0]  q;
reg [counter_bits-1:0]  q=0;
assign  co=(q==(n-1)) && en; //进位
always @(posedge clk)
begin
    if(r) q=0;
    else if(en) //en=1,计数  en=0,保持
        begin if(q==(n-1))  q=0 ; else q=q+1; end
end
endmodule

```

(3) 测试代码 (testbench)

```

`timescale 1ns/100ps
module FlowingWaterLight_tb;
parameter DELY=10;
reg  clk, reset, direction;
wire [7:0]  led;
//顶层模块实例，传递 sim 参数 1，改变分频比，减少仿真时间
FlowingWaterLight #(sim(1)) FlowingWaterLight_inst (
    .clk(clk),.reset(reset),.direction(direction),.led (led));
//产生时钟波形
initial begin clk = 0; forever #5 clk = ~clk;end
///产生 reset、 direction 波形
initial  begin
    reset = 0; direction = 1;
    #(DELY/2+1)reset = 1;
    #(DELY)reset = 0;
    #(DELY*80) direction = 0;
    #(DELY*90)$stop;  end
endmodule

```

五、实验步骤

Vivado 设计分为 Project Mode 和 Non-project Mode 两种模式，一般简单设计中，常用的是 Project Mode。在本实验中，我们将以一个简单的实验案例，一步一步地完成 Vivado 的整个设计流程。

1. 下载文件

(1) 从网站中下载 labA4_FlowingWaterLight 文件夹至硬盘里，存放在 e:\SystemDesignLab 文件下。labA4_FlowingWaterLight 共二个子文件夹，其中 src 和 vivado 二个文件夹分别存放源代码和 Vivado 工程。

src 有顶层设计代码 FlowingWaterLight.v、任意进制计数器/分频器设计代码 conter_n.v、任意位数可逆计数器设计代码 conter_up_down.v、译码器模块设计代码 decode.v 和顶层测试

代码 FlowingWaterLight_tb.v。为了更好地学习 Vivado 软件使用方法，在这些文件中，作者人为地设置了三处错误。

2. 建立 Vivado 工程文件

(1) 启动 Xilinx Vivado 系统软件，可通过桌面 Vivado 2017.1 快捷方式或开始菜单中 Xilinx Design Tools Vivado 2017.1 下的 Vivado 2017.1 打开软件，开启后，软件如图 A4.20 所示。

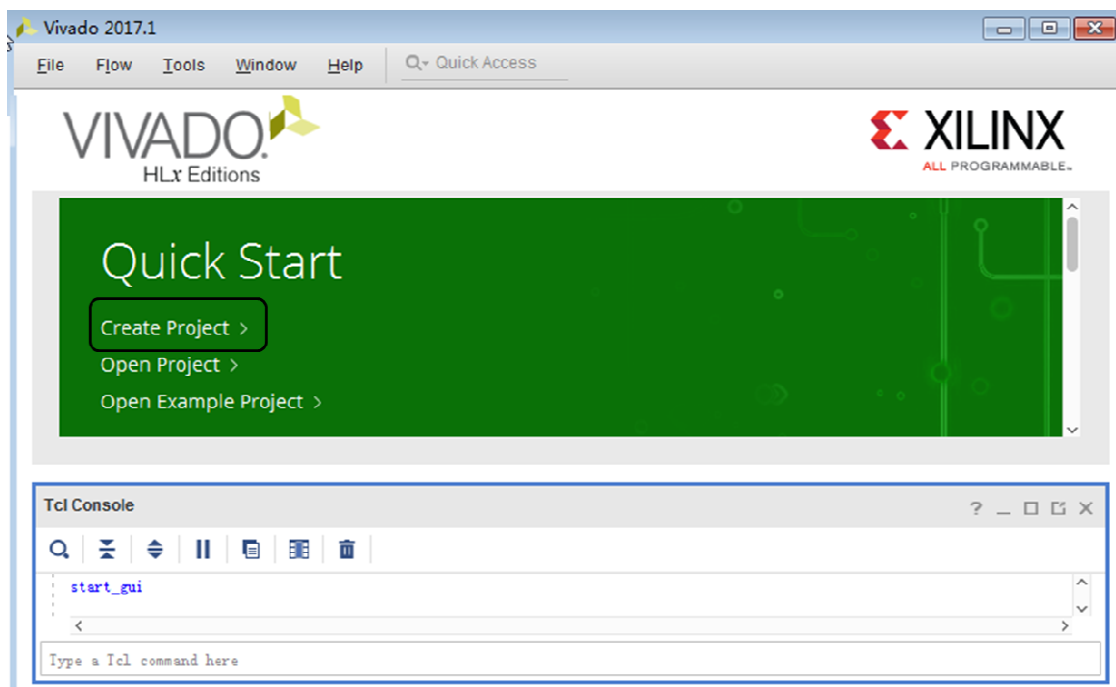


图 A4.20 Vivado 启动界面

(2) 单击上述界面中 Create Project 图标，弹出新建工程向导，如图 A4.21 所示，点击 Next，进入填写工程名界面，如图 A4.22 所示。

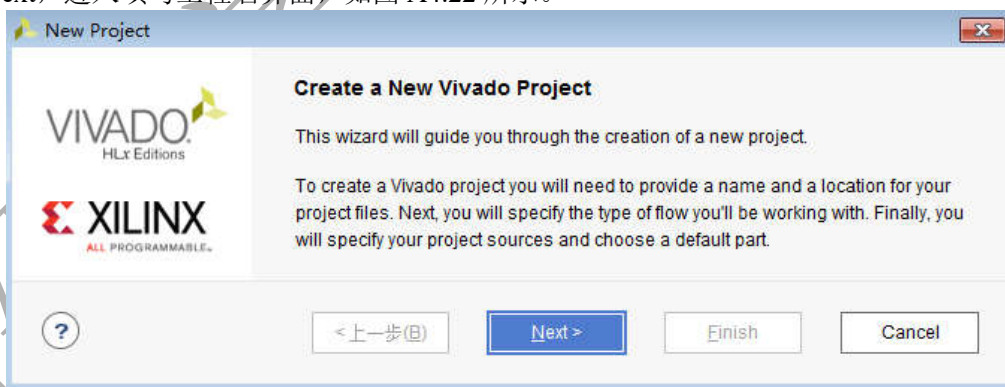


图 A4.21 Vivado 新建工程向导之一

(3) 在图 A4.22 中，输入 Vivado 工程名称：FlowingWaterLight；选择工程存储路径：E:/SystemDesignLab/labA4_FlowingWaterLight/vivado；不要勾选 Create project subdirectory 选项。设置完成后，点击 Next，进入工程类型选择界面。

注意：工程名称和存储路径中不能出现中文和空格，建议工程名称以字母、数字、下划线来组成。



图 A4.22 Vivado 新建工程向导之二

(4) 在图 A4.23 中, 选择 RTL Project 一项, 并勾选 Do not specify sources at this time, 勾选该选项目的是为了跳过在新建工程的过程中添加设计源文件。点击 Next, 进入 FPGA 目标器件选择界面。

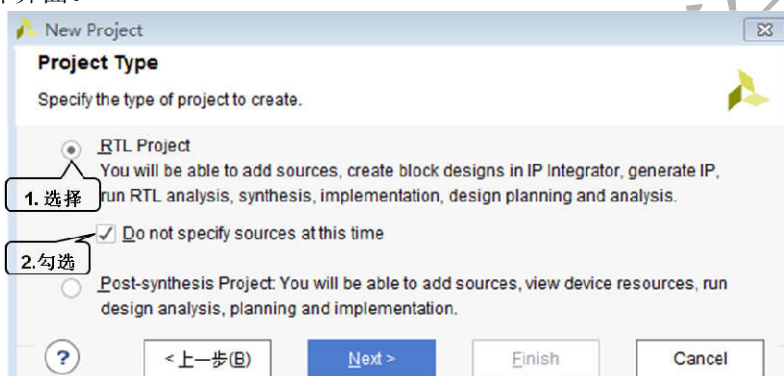


图 A4.23 Vivado 新建工程向导之三

(5) FPGA 目标器件与使用的 FPGA 开发平台有关, 本教程将用到两种型号的 FPGA 开发板: Nexys Video 和 Basys 3; FPGA 目标器件分别为 Artix-7 xc7a200tfg484-1 和 Artix-7 xc7a35tcbg236-1。教程将以 Nexys Video 开发板为主介绍开发过程。

图 A4.24 为 Nexys Video 开发板的 FPGA 目标器件选择的示意图, 依次选 Family 为 Artix-7, 封装形式 (Package) 为 fbg484, 速度等级 (Speed grade) 为-1。并在 part 列表中选择 xc7a200tfg484-1, 点击 Next, 进入信息确认界面。

注意: 若使用 Basys 3 开发板, 则需选择 Artix-7 xc7a35tcbg236-1 器件: Family 为 Artix-7, 封装形式 (Package) 为 cpg236, 速度等级 (Speed grade) 为-1。

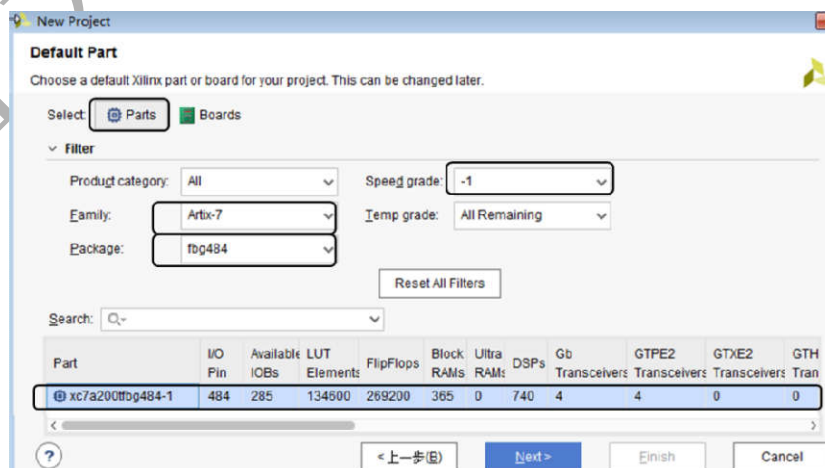


图 A4.24 Vivado 新建工程向导之四

(6) 在图 A4.25 中, 确认相关信息与设计所用的的 FPGA 器件信息是否一致, 一致请点击 Finish, 完成空白工程的创建, 如图 A4.26 所示。

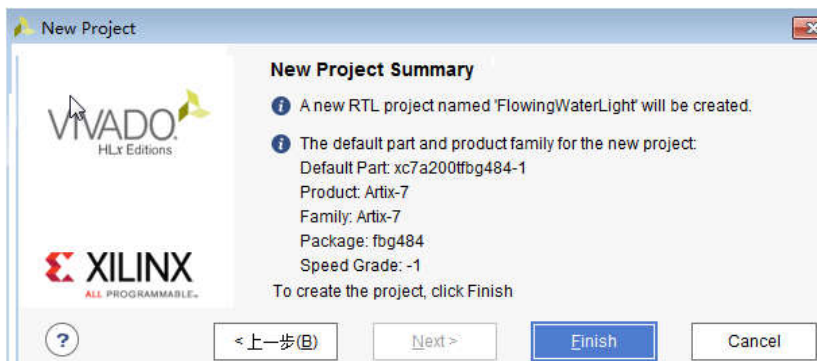


图 A4.25 Vivado 新建工程向导之五

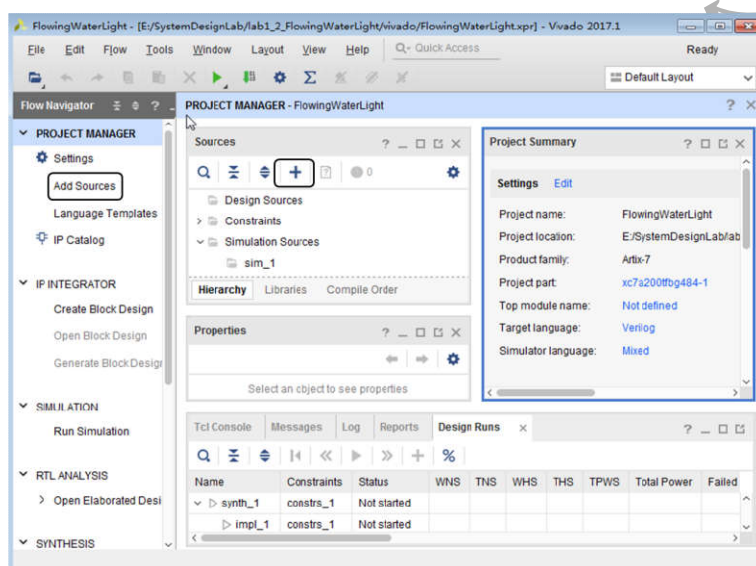


图 A4.26 新建工程完成后的界面

3. 给工程加入设计文件和仿真文件

(1) 在图 A4.26 所示工程界面, 在 Flow Navigator 窗口中, 点击 PROJECT MANAGER⇒Settings⇒Add Sources 命令, 打开设计文件导入添加对话框, 如图 A4.27 所示。选择第二项 Add or create Design Sources, 用来添加或新建 Verilog 或 VHDL 源文件, 点击 Next, 进入图 A4.28 所示界面。

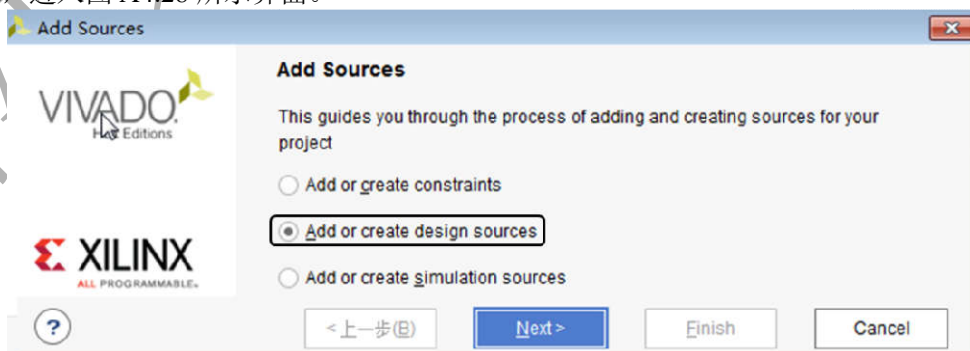


图 A4.27 设计文件导入添加对话框

(2) 在图 A4.28 界面, 通过点击 Add Files 按钮, 根据添加文件导航选择需加入的设计文件。本例需要加入 FlowingWaterLight.v、dffr.v、counter_n.v counter_up_down.v 和 decode.v

共 4 个文件。这些文件均存在文件夹 E:\SystemDesignLab\labA4_FlowingWaterLight\src 下。注意，每次可选择多个文件。

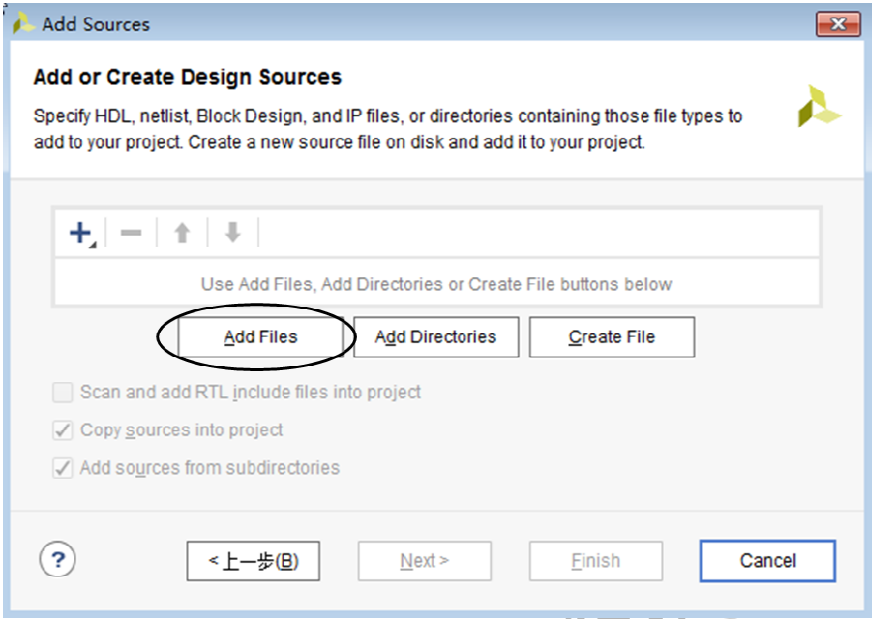


图 A4.28 导入设计文件对话框之一

(3) 选择添加文件后的界面如图 A4.29 所示，点击 **Finish** 按钮，即可完成文件的导入。因为有两个开发软件 ModelSim 和 Vivado 都可能修改设计文件，必须将代码保留在 src 文件夹里，所以图 A4.29 界面中的 Copy Sources into project 这一选项千万不要勾选。

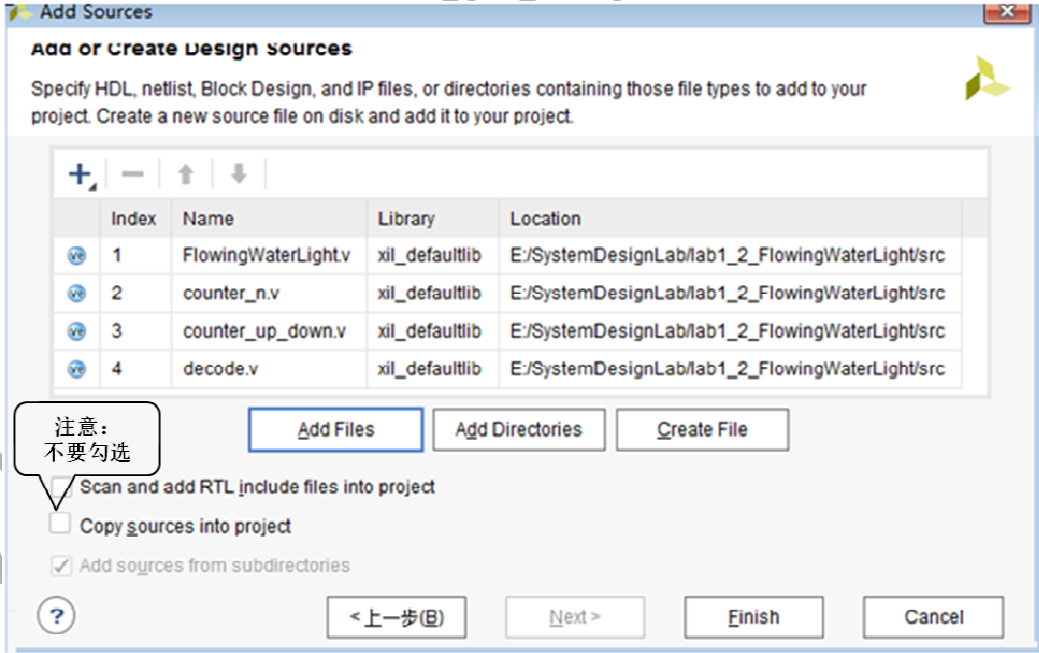


图 A4.29 导入设计文件对话框之二

(4) 重复 (1) ~ (3) 步骤，加入仿真测试文件 FlowingWaterLight_tb.v，只是在图 A4.27 中选中第 3 项 Add or create simulation Sources。

导入文件后，Sources 窗口列出设计文件层次结构，如图 A4.30 所示。图中显示 decode.v 有语法错误。另外，实例名 div_inst 前的“？”表示调入分频器模块失败。

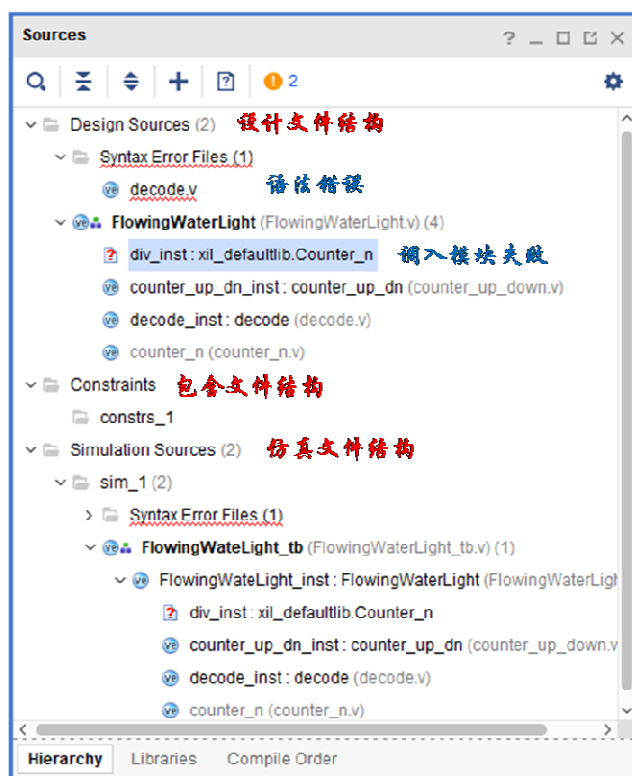


图 A4.30 设计文件的层次结构

4. 修改语法错误和模块调用错误

(1) 在图 A4.30 中，双击 decode.v 文件名。在打开的代码窗口中，如图 A4.31 所示，vivado 会在错误处用红色波浪下划线标记。很明显，只需代码的第 8 行逗号改为分号即可。

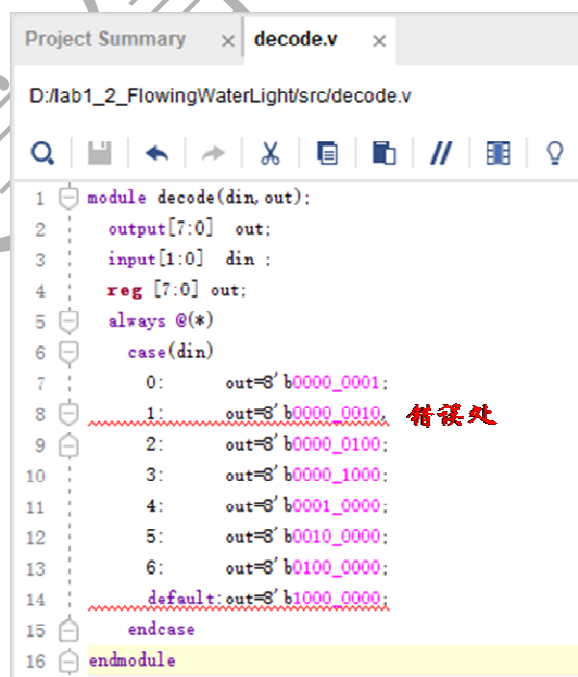


图 A4.31 decode.v 文件

(2) 从图 A4.30 可明显看出，顶层 FlowingWaterLight 调用 Counter_n 模块失败。但文

件结构中却有一个 `counter_n` 模块，显然是 `FlowingWaterLight` 调用计数器模块时，模块名大小写弄错了。修改 `FlowingWaterLight.v` 代码，即可得到如图 A4.32 所示的正确的设计文件的层次结构。

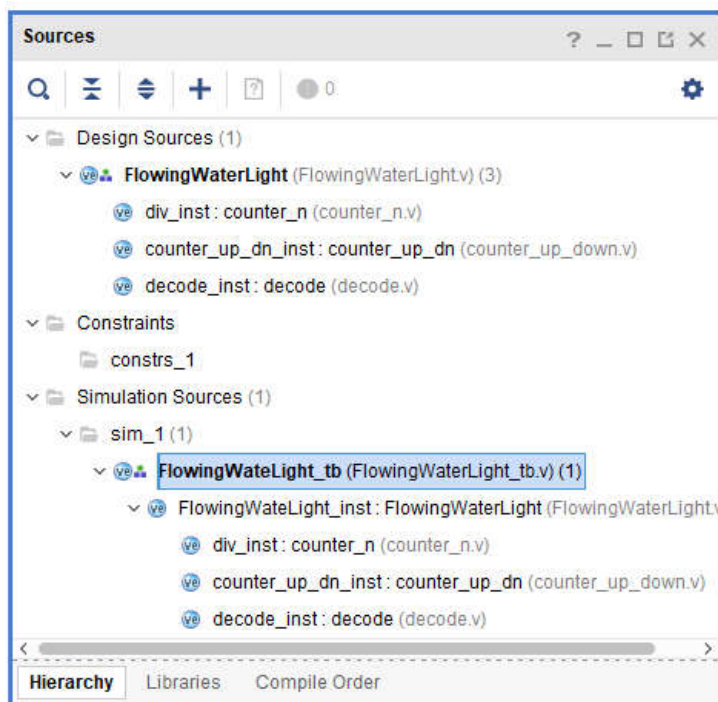


图 A4.32 设计文件的层次结构

5. 代码仿真

(1) 在源文件代码中, 选中仿真测试文件 FlowingWaterLight_tb.v。然后在 Flow Navigator 窗口下, 点击 SIMULATION⇒Run Simulation⇒ Run Behavioral Simulation 命令, Vivado 将会打开仿真窗口并运行 1000ns,如图 A4.33 所示。

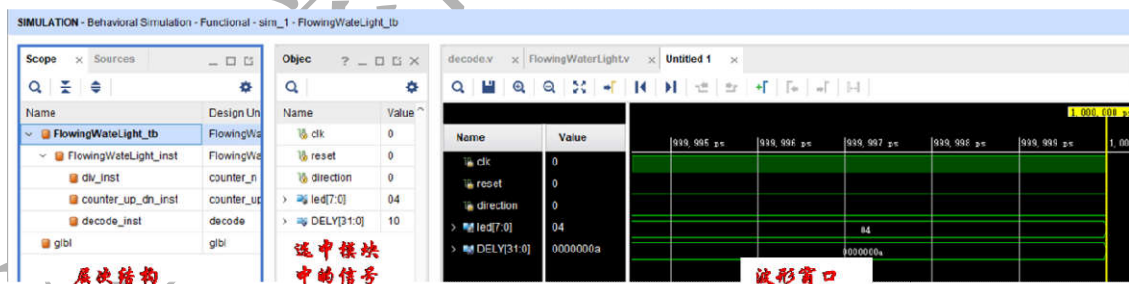


图 A4.33 仿真窗口

(2) 设置信号的编码方式

信号的默认显示方式为 16 进制，为了增加仿真结果的可读性，本例中 led 信号用二进制编码方式较为合适。在图 A4.33 波形窗口右击 led[7:0]信号，在弹出的快捷菜单执行 Radix⇒Binary 命令，将 led 信号编码方式改为二进制。

(3) 执行仿真

选择菜单 **Run⇒Run-ALL** 命令，得到图 A4.34 所示的仿真波形（由于篇幅原因，只显示部分波形），从仿真波形可看出，led 高 4 位始终为 0，设计存在错误。下面将通过调试手段，查找设计错误。

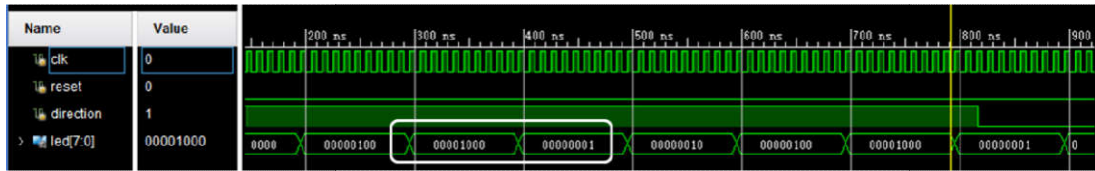


图 A4.34 仿真波形之一

首先，观察顶层设计的中间变量 pulse2Hz 和 q 的波形，以确定设计错误发生在哪个模块，方法如下。

①如图 A4.35 所示，在文件层次结构区域中，选中顶层模块 FlowingWaterLight，可在 Objects 窗口内选定要观察的信号 pulse2Hz 和 q，通过右键快捷菜单的 Add Wave 命令加到波形 Wave 窗口。也可直接将 Objects 窗口中的信号“拖至”波形 Wave 窗口。

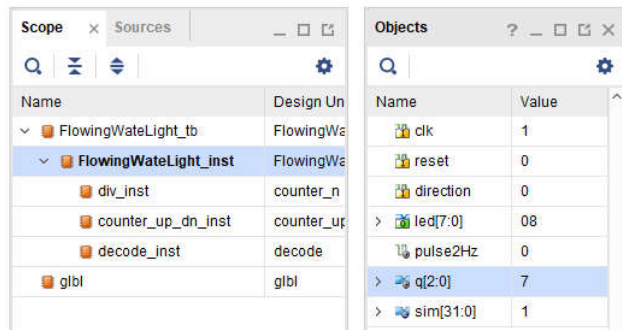


图 A4.35 加入顶层设计的信号方法

②执行菜单 Run⇒Restart 命令。

③执行菜单 Run⇒Run-ALL 命令，再次仿真。

仿真波形如图 A4.36 所示，从仿真波形可知，pulse2Hz 和 q 波形均正确，基本可确定设计错误发生在 decode 模块。

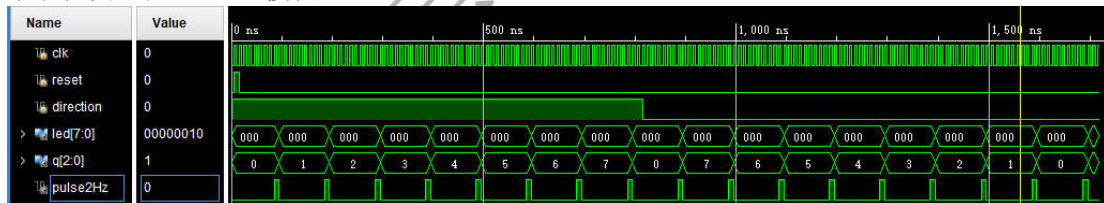


图 A4.36 仿真波形之二

其次，根据上面分析，加入 decode 模块的全部信号，重新仿真，得到如图 A4.37 的仿真波形。仔细分析波形，可看出 decode 模块的 din 信号出错（din 信号应与顶层的 q 信号一致），主要是 din 信号位宽不对。

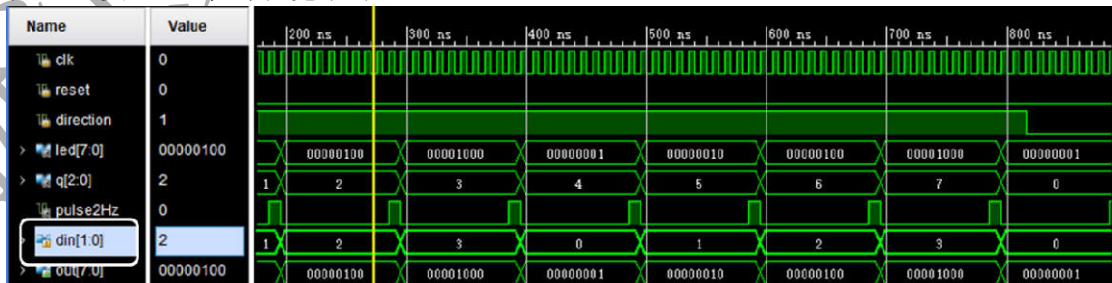


图 A4.37 仿真波形之三

最后，打开 decode.v 文件，将第 6 行语句“input[1:0] din;”修改为“input[2:0] din;”。保存修改后的 decode.v 文件；执行菜单 Run⇒Relaunch Simulation 命令；执行菜单

Run⇒Run-ALL 命令，重新仿真。得到如图 A4.38 的仿真波形。

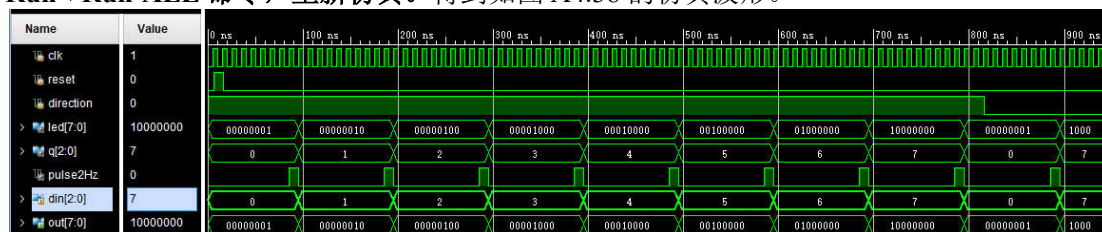


图 A4.38 最后的仿真结果

分析图 A4.38 所示的仿真波形，仿真结果符合设计要求，至此，功能仿真结束。

注意，当修改代码后，必须执行菜单 Run⇒Relaunch Simulation 命令。

6. 电路代码综合

在 Flow Navigator 窗口下，点击 SYNTHESIS ⇒Run Synthesis 命令，Vivado 将对工程进行综合。综合完成之后，在弹出的如图 A4.39 所示的窗口中选择 Open Synthesized Design 选项并点击 OK 按钮，打开综合结果。

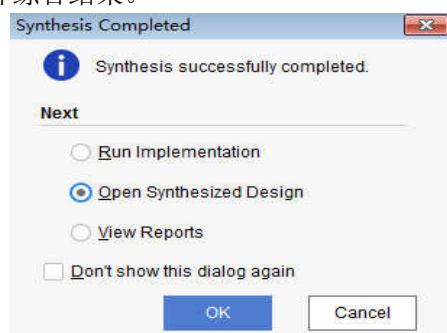


图 A4.39 综合完成后界面

对初学者来说，综合结果中的原理图较为重要。可在图 A4.40 的 Layout 下列表框选择 Debug 选项打开综合后的原理图。

小提示：在原理图，点击模块左上角的“+”可展开该模块的底层设计，同样，点击“-”收起底层设计。

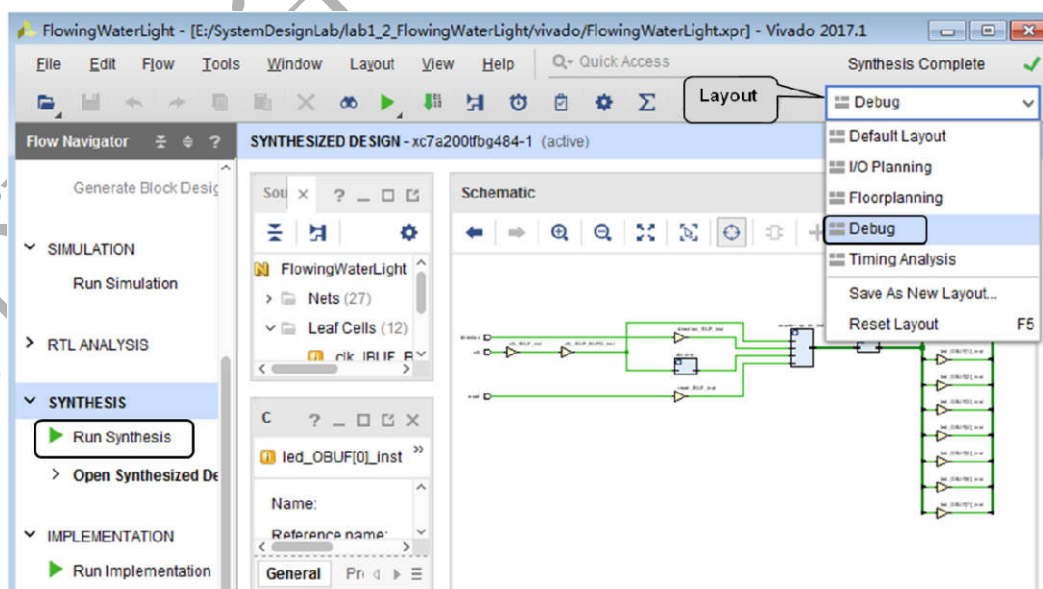


图 A4.40 综合后的原理图

原理图有助于查看模块之间的连接,也可检查 HDL 语言描述的电路是否符合设计要求,因此,原理图是查找设计错误的好帮手。

7. 约束

约束可以分为时钟约束、I/O 约束以及时序例外约束三大类,约束目的是:提高设计的工作频率;获得正确的时序分析报告;指定 FPGA 引脚位置与电气标准。由于篇幅限制,本例只对引脚分配和主时钟的周期进行附加约束。

有两种方法可以添加约束文件,一是在图形界面进行设置;二是直接新建 XDC 约束文件,手动输入约束命令。为使初学者更好地理解电路概念,作者采用第一种方法。

(1) 引脚约束

引脚分配由 FPGA 开发来决定,表 4.1 列出两种开发板的引脚分配方法,可根据自己手上的开发板来选择。

表 4.1FPGA 引脚约束内容

I/O 名称	引脚编号 (Nexvs)	引脚编号 (Basvs)	电气标准	说明
clk	R4	W5	LVCM	系统 100MHz 主时钟信号
reset	B22	U18	LVCM	中间按键 (BTNC)
directio	E22	V17	LVCM	逻辑开关 SW0
Led[0]	T14	U16	LVCM	LED 指示灯
Led[1]	T15	E19	LVCM	
Led[2]	T16	U19	LVCM	
Led[3]	U16	V19	LVCM	
Led[4]	V15	W18	LVCM	
Led[5]	W16	U15	LVCM	
Led[6]	W15	U14	LVCM	
Led[7]	Y13	V14	LVCM	

①如图 A4.41 所示,在工程界面中 Layout 列下列表框中选择 IO planning 选项,并在下方的选项卡中切换到 I/O ports 页面,I/O ports 窗口下将罗列出设计的所有输入/输出引脚。

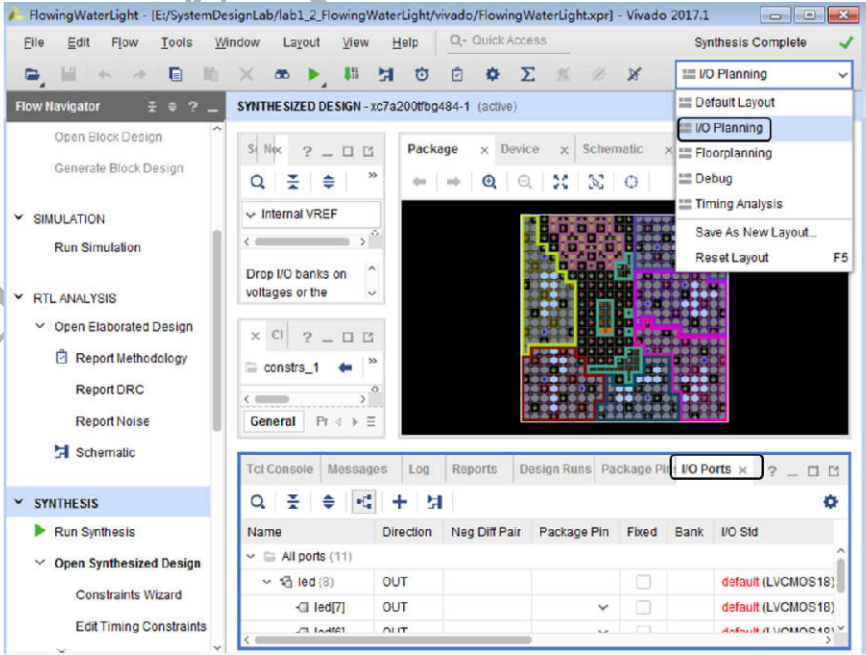


图 A4.41 引脚约束之一

②如图 A4.42 所示，在 I/O ports 页面，展开引脚。对于每个输入或输出引脚，在 Package Pins 栏填写或选择对应引脚编号；并在 I/O std 栏下拉选择引脚的电气标准，引脚的电气标准常采用 LVCMOS33。当引脚约束没问题时，相应 Fixed 栏将处于勾选状态。

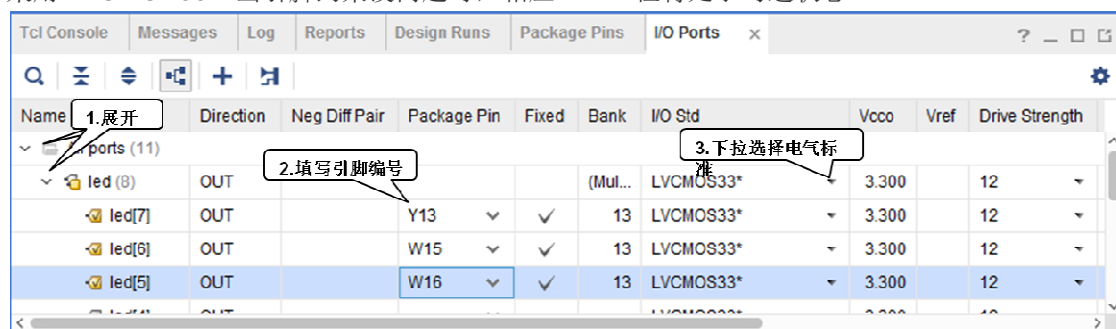


图 A4.42 引脚约束之二

③完成所有引脚约束之后，点击工具栏中的保存按钮，在图 A4.43 所示的界面中输入约束文件的文件名：FlowingWaterLight；选择存储路径，一般存储在源文件夹 src 中。然后，点击 OK 完成引脚约束过程。

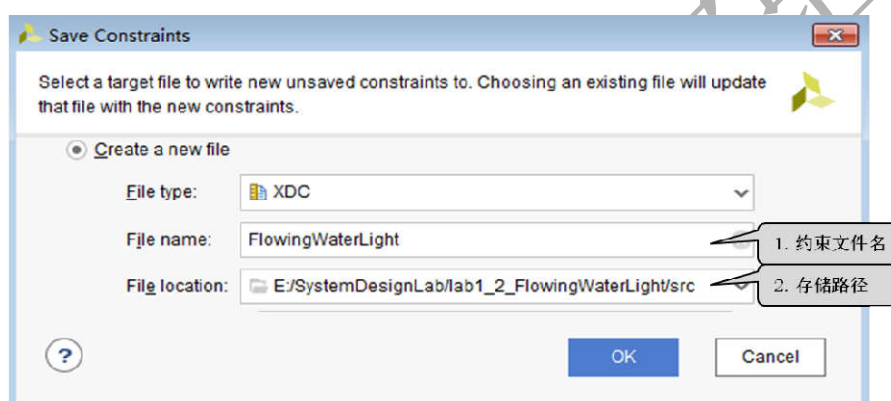


图 A4.43 填写约束文件名

完成引脚约束后，在 Sources 窗口下 Constraints 中会找到的 XDC 文件，如图 A4.44 所示，双击 XDC 文件，可在 Text Editor 窗口打开查看和编辑 XDC 文件内容。

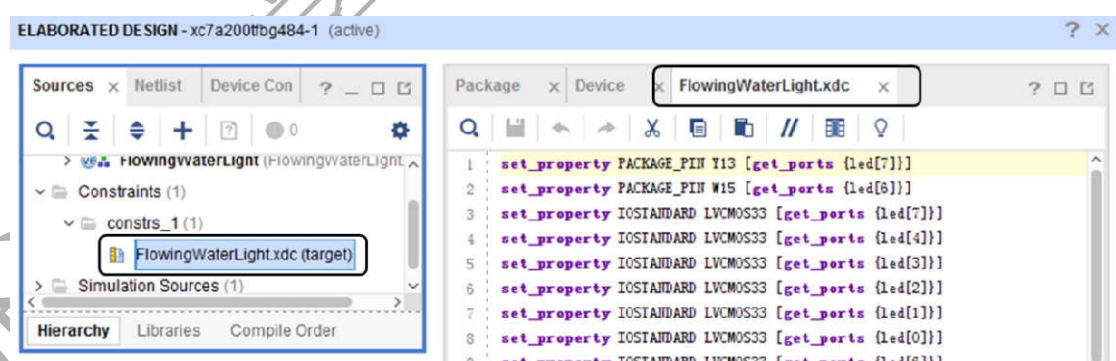


图 A4.44 约束文件及其内容

(2) 时钟约束

本例仅对主时钟的周期和占空比进行附加约束，开发板输入主时钟频率为 100MHz。方法如下。

在 Flow Navigator 窗口下，点击 SYNTHESIS ⇒ Open Synthesis Design ⇒ Constraints ⇒ Constraints Wizard 命令，在弹出的 Timing Constraints Wizard 对话框中点击 Next 按钮，然

后在图 A4.45 所示的界面中填写时钟频率（100MHz），由于占空比 50%，因此其它参数采用默认。点击 Skip to Finish 按钮后，再在弹出的 Timing Constraints Wizard 对话框中点击 Finish 按钮，即可完成时钟约束。

完成时钟约束后，约束文件 FlowingWaterLight.XDC 的最末添加了一行时钟约束语句：

```
create_clock -period 10.000 -name clk -waveform {0.000 5.000} [get_ports clk]
```

小提醒：如果进行时钟约束时，没有关闭约束文件 FlowingWaterLight.XDC，此时需要 Reload 约束文件才能看到添加的时钟约束语句。

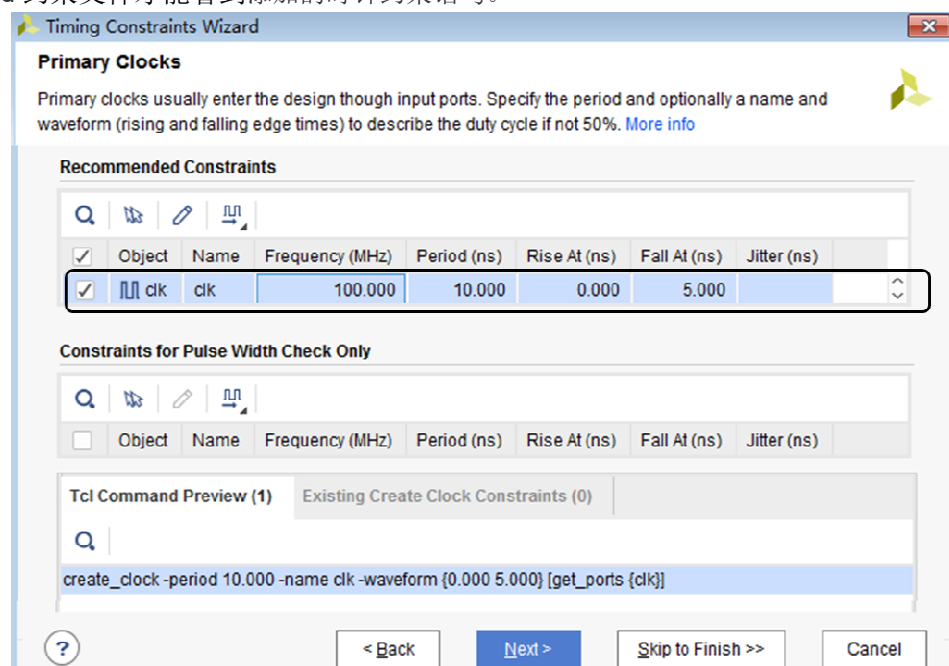


图 A4.45 时钟约束

8. 工程实现

设计实现是指将综合输出的网表适配到 FPGA 上，方法如下。

在 Flow Navigator 窗口下，点击 IMPLEMENTATION⇒ Run Implementation 命令，在弹出的 Launch Runs 窗口，直接点击 OK 按钮进行实现工作。由于进行约束后没有重新综合，所以 Vivado 会依次完成综合、实现两个过程。

实现完成后，在如图 A4.46 所示的界面中，可选择第二选项 Generate Bitstream，然后点击 OK 按钮，直接进行编程比特流生成工作。也可点击 Cancel 按钮取消。

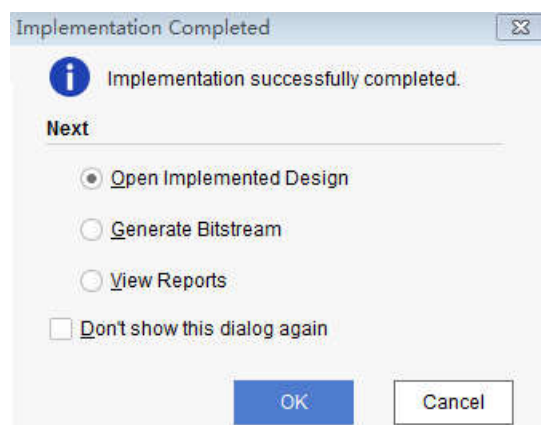


图 A4.46 设计实现完成界面

9. 生成 FPGA 编程比特流

FPGA 编程比特流的生成可从设计实现完成后的界面直接进入。也可在 Flow Navigator 窗口下，点击 PROGRAM AND DEBUG⇒Generate Bitstream 命令，然后在弹出的 Launch Runs 窗口，直接点击 OK 按钮的进入。

完成后，在弹出的界面也可点击 Cancel 按钮。

10. 编程下载

(1) 用 USB 线将开发板与计算机的 USB 接口连接。

(2) 用 3.3V 的 DC 电源适配器给 Nexys Videog 开发板供电。Basys3 开发板由 USB 供电，所以，如使用 Basys3 开发板，则不需外加电源供电。

(3) 打开开发板电源。

(4) 在 Flow Navigator 窗口下，点击 PROGRAM AND DEBUG⇒Open Hardware Manger 命令。在图 A4.47 所示的 HARDWARE MANGER 对话框中，点击 OpenTarget 后，再点击 Auto Connect 命令。Vivado 会自动查找 JTAG chain 上的设备。连接完成，FPGA 芯片会出现在 Hardware 窗口里，如图 A4.48 所示。

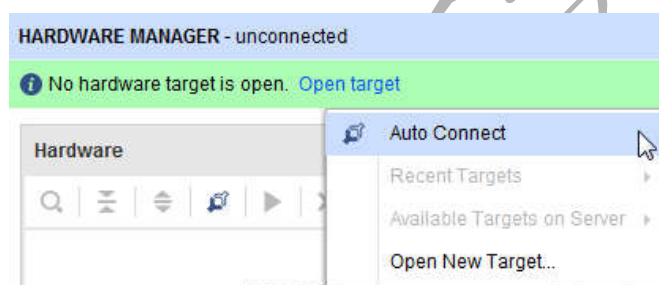


图 A4.47 配置对话框之一

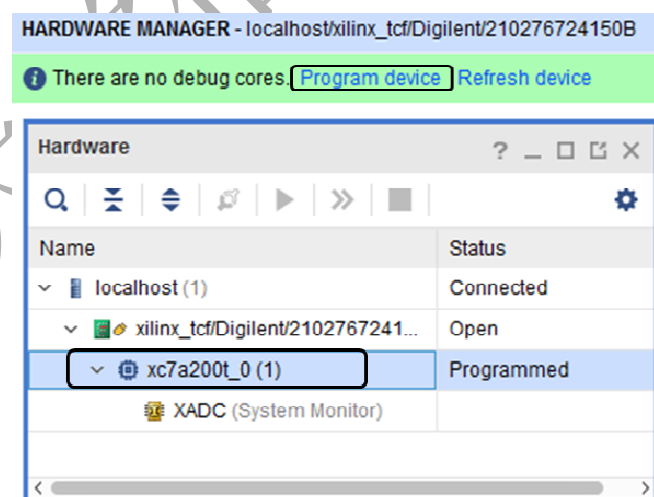


图 A4.48 配置对话框之二

(5) 点击图 A4.48 界面中的 Program device, 然后在弹出如图 A4.49 所示 Program Device 窗口中点出 Program 即可完成下载编程。

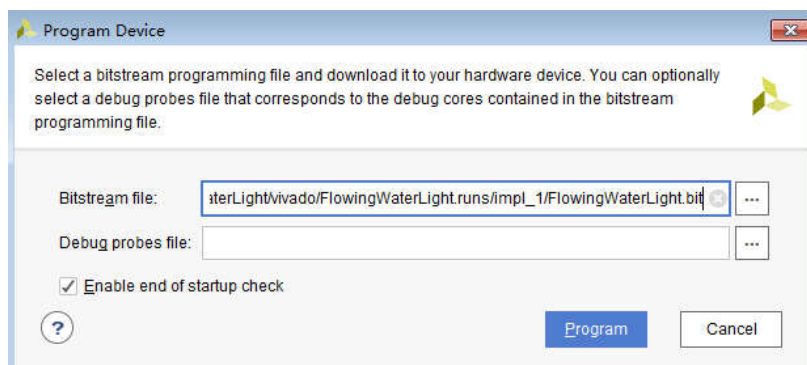


图 A4.49 下载

11. 在实验板验证实验结果

根据 I/O 引脚约束内容，在开发实验板上拨动相应的开关、按键，观看流水灯是否符合设计要求，验证设计是否成功。

至此，整个流水灯实验完成任务。

六、思考题

- (1) Vivado 的开发流程主要有哪几个步骤？各个步骤主要作用是什么？
- (2) 附加约束的主要目的是什么？时序约束的主要种类有哪些？
- (3) Vivado 集成开发工具自带仿真器，为什么仍通常要求使用第三方仿真软件 ModelSim？